

API de Governança e Compliance Ambiental

Cidades ESG Inteligentes · Automação completa do ciclo de vida com CI/CD, Docker e Kubernetes

Integrante	Gustavo Silva Torres — RM564909
Tema ESG	Cidades ESG Inteligentes — Governança e compliance ambiental
Aplicação base	Web API em C# / ASP.NET Core 8 (desafio anterior)
Ferramenta CI/CD	GitHub Actions + GitHub Container Registry
Containerização	Docker multi-stage · Docker Compose · Kubernetes (Kustomize)
Ambientes	Staging (porta 8081) e Produção (porta 8080)
Repositório	github.com/gusttores/DEVOPS_FIAP

1. Visão geral do projeto

A aplicação é uma **API RESTful de governança e compliance ambiental**, construída em ASP.NET Core 8 no desafio anterior. Ela registra normas ambientais, planeja auditorias internas e executa um **motor de avaliação automática**: ao receber as evidências de uma auditoria, calcula um score ponderado de conformidade, classifica o resultado e gera as não-conformidades com seus planos de ação e prazos.

Nesta fase, o objetivo deixa de ser a regra de negócio e passa a ser o **ciclo de vida da aplicação**: garantir que qualquer mudança de código seja compilada, testada, empacotada e implantada em dois ambientes **sem intervenção manual** — com um único ponto de decisão humana, a aprovação para produção.

1.1 O que a aplicação entrega

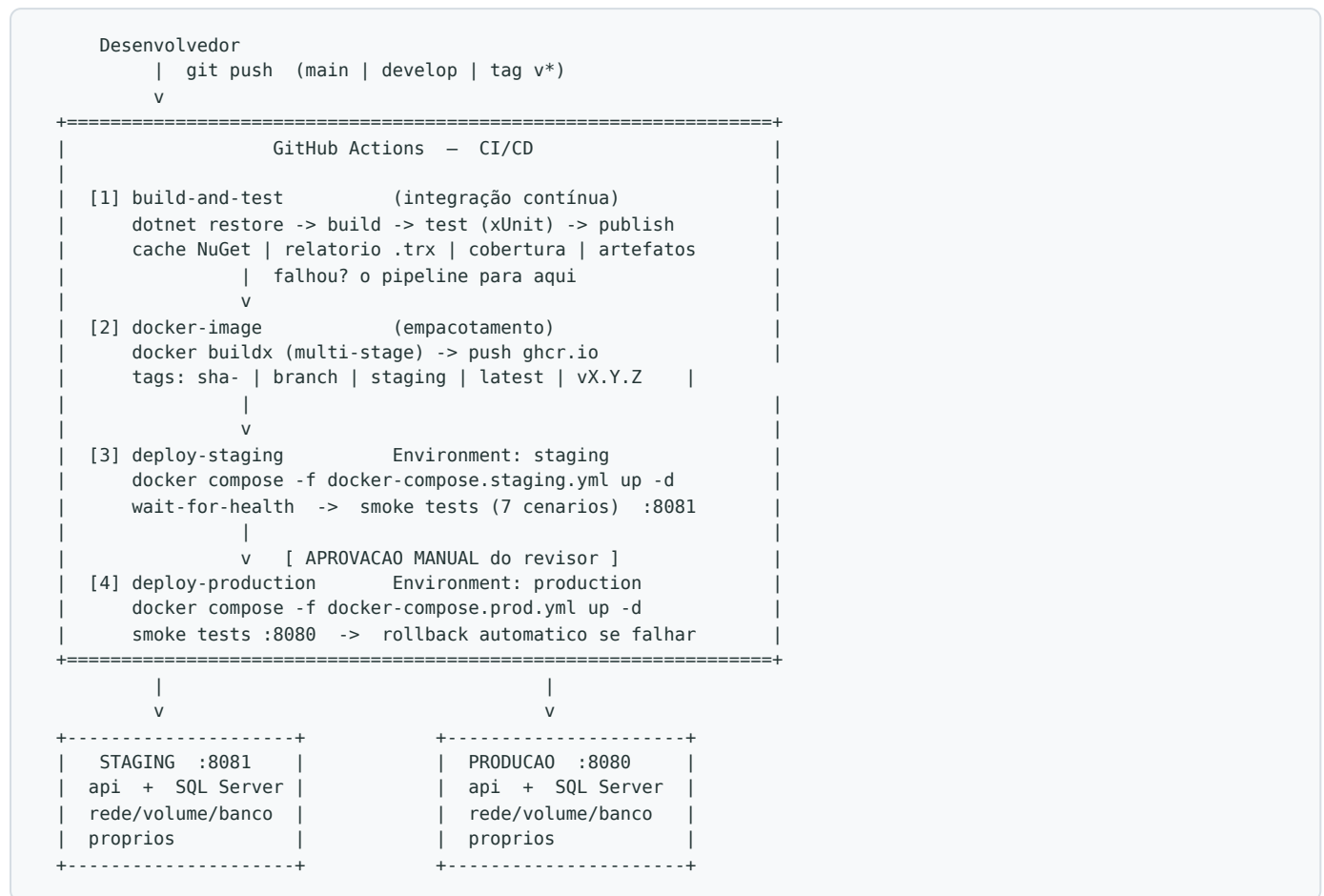
Característica	Implementação
Endpoints REST	9 endpoints em 4 controllers (Auth, Normas, Auditorias, Conformidade)
Arquitetura	MVVM — Models · ViewModels · Controllers / Services
Segurança	JWT Bearer com [Authorize(Roles="Auditor")] nos endpoints críticos
Validação e erros	FluentValidation + filtro global · middleware de exceções → ProblemDetails
Persistência	EF Core 8 + SQL Server 2022, migrations aplicadas no startup
Observabilidade	/health (liveness) e /health/ready (readiness com checagem do banco)
Testes	25 testes de integração xUnit com WebApplicationFactory + EF Core InMemory

1.2 O que foi adicionado nesta entrega

- **Pipeline CI/CD** em GitHub Actions com 4 jobs encadeados: build, testes, imagem Docker e deploy em dois ambientes.
- **Dockerfile multi-stage** reescrito: stage de testes, usuário não-root, HEALTHCHECK e versionamento da imagem.
- **Três arquivos Docker Compose** (desenvolvimento, staging e produção) com volumes, redes e variáveis de ambiente próprios.
- **Manifests Kubernetes** em Kustomize (base + overlays de staging e produção) com PVC, ConfigMap, Secret, Ingress, HPA e PDB.
- **Health checks** na aplicação, consumidos pelo Docker, pelo Compose e pelas probes do Kubernetes.
- **Smoke tests** pós-deploy em Bash, que validam o ambiente já publicado.

2. Arquitetura da solução DevOps

O fluxo abaixo descreve o caminho percorrido por um commit até estar rodando em produção. A promoção entre etapas é automática, exceto pelo portão de aprovação manual antes de produção.



2.1 Princípios aplicados

Princípio	Como aparece na solução
Build once, deploy many	A imagem é construída uma única vez e a mesma imagem (mesmo digest) é implantada em staging e produção. O que muda é só a configuração injetada por variável de ambiente.
Configuração fora do código	Nenhum segredo na imagem: connection string, chave JWT e credenciais vêm de variáveis de ambiente (12-factor), de GitHub Secrets no pipeline e de <code>Secret</code> no Kubernetes.
Fail fast	Testes rodam antes do empacotamento; se falharem, nenhuma imagem é publicada e nenhum deploy acontece.
Validação em produção-like	Staging é idêntico a produção em imagem e topologia, mudando apenas porta, recursos e nível de log.
Reversibilidade	Toda imagem é versionada por commit (<code>sha-abc1234</code>); o rollback é trocar a tag e subir de novo — automatizado no job de produção.
Infraestrutura como código	Compose, manifests Kubernetes, workflows e scripts versionados junto do código-fonte.

3. Descrição do pipeline

3.1 Ferramenta escolhida e justificativa

GitHub Actions, com o GitHub Container Registry (GHCR) como registry de imagens. A escolha se apoia em quatro pontos:

- **Sem infraestrutura para manter** — diferente do Jenkins, não há servidor, plugins ou agentes a administrar; o runner é efêmero e limpo a cada execução.
- **Registry integrado** — o GHCR autentica com o `GITHUB_TOKEN` gerado automaticamente, sem cadastrar credenciais externas.
- **Environments com aprovação** — o recurso de *required reviewers* entrega o portão manual de produção nativamente, sem script.
- **Evidência automática** — logs, relatórios de teste e artefatos ficam anexados a cada execução, o que atende diretamente ao requisito de comprovação da entrega.

3.2 Gatilhos

Evento	O que é executado
<code>push</code> em <code>develop</code>	build + testes + imagem + deploy em staging
<code>push</code> em <code>main</code>	tudo acima + deploy em produção (após aprovação)
<code>push</code> de tag <code>v*</code>	idem <code>main</code> , com a imagem versionada (ex.: <code>v1.0.0</code>)
<code>pull_request</code> para <code>main</code>	somente build + testes — não publica imagem nem implanta
<code>workflow_dispatch</code>	execução manual pelo botão <i>Run workflow</i>

3.3 Etapas e lógica

Etapa 1 — build-and-test · integração contínua

- Instala o .NET 8 SDK e restaura o cache de pacotes NuGet (chave pelo hash dos `.csproj`).
- `dotnet restore` → `dotnet build -c Release`.
- `dotnet test` executa os 25 testes xUnit gerando relatório `.trx` e cobertura de código.
- O relatório é publicado no resumo do job e salvo como artefato `resultados-testes`.
- `dotnet publish` gera os binários, salvos como artefato `app-publish`.
- **Lógica:** este job é o portão de qualidade. Qualquer falha interrompe o pipeline antes de qualquer publicação.

Etapa 2 — docker-image · empacotamento

- Configura o Buildx e autentica no GHCR com o `GITHUB_TOKEN`.
- `docker/metadata-action` gera as tags automaticamente: `sha-<commit>`, nome da branch, `staging`, `latest` e a versão semântica quando há tag.
- Build multi-stage com cache de camadas do próprio Actions (`type=gha`) e push da imagem.
- Digest e versão publicados no resumo do job.
- **Lógica:** o artefato implantável passa a ser a imagem imutável, identificada pelo commit que a originou.

Etapa 3 — deploy-staging · entrega contínua

- Usa o GitHub Environment `staging`.
- Monta o `.env.staging` a partir dos secrets do repositório (com valores padrão de estudo como *fallback*).
- Sobe a stack com `docker compose -f docker-compose.staging.yml up -d` usando a imagem recém-publicada.
- `scripts/wait-for-health.sh` aguarda `/health` responder 200 (até 240 s).
- `scripts/smoke-test.sh` valida 7 cenários reais no ambiente no ar.
- Logs, `docker compose ps` e o JSON do health são salvos como artefato de evidência.
- **Lógica:** staging prova que a imagem sobe e responde de verdade — não apenas que compila.

Etapa 4 — deploy-production · deploy contínuo com portão

- Depende do sucesso de staging e só roda em `main` ou em tags `v*`.
- O Environment `production` está configurado com *required reviewers*: o job fica em *Waiting* até a aprovação manual.
- Sobe `docker-compose.prod.yml` na porta 8080 e repete os smoke tests.
- **Rollback automático:** se os smoke tests falharem, o passo `if: failure()` recoloca a tag `latest` (última imagem estável) e recria o container.
- Evidências retidas por 30 dias.

3.4 Os 7 cenários dos smoke tests

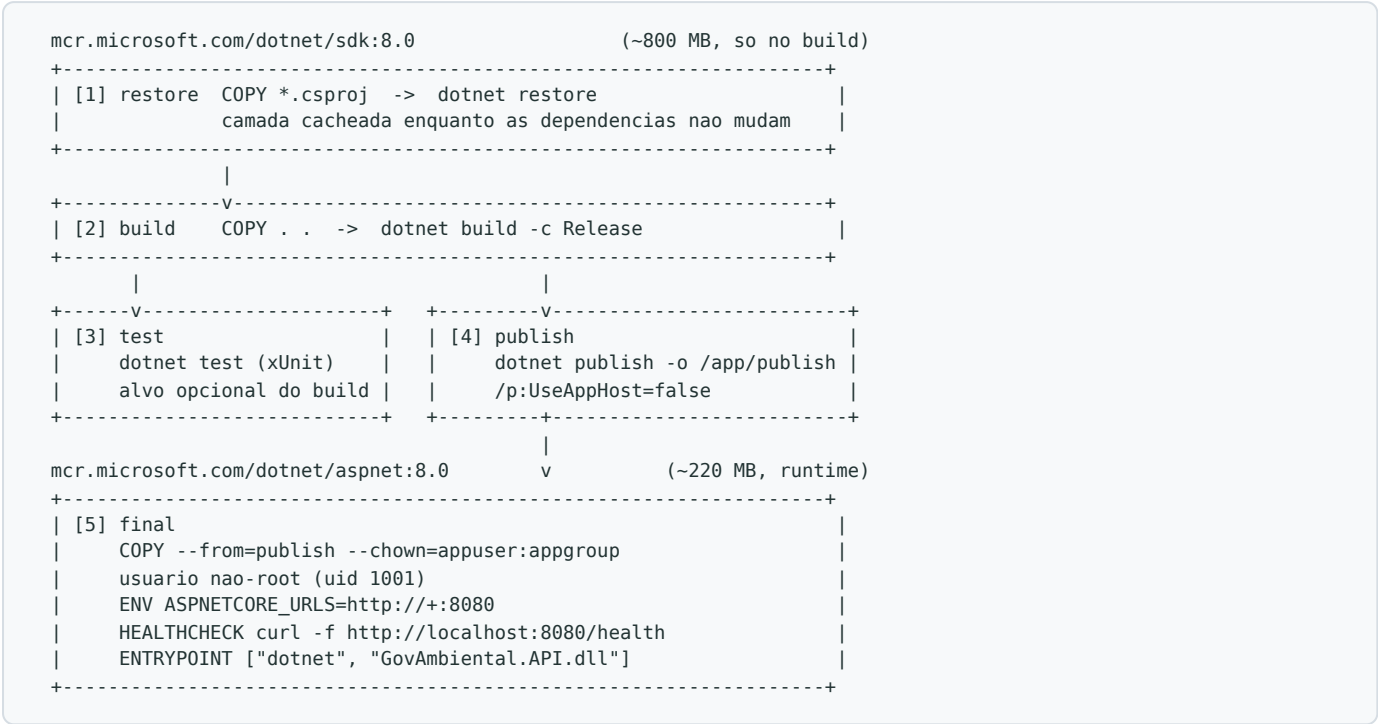
#	Verificação	Esperado
1	<code>GET /health</code> — liveness	HTTP 200
2	<code>GET /health/ready</code> — readiness com banco	HTTP 200
3	<code>GET /swagger/v1/swagger.json</code> — contrato OpenAPI	HTTP 200
4	<code>GET /api/normas?pageNumber=1&pageSize=5</code> — listagem paginada	HTTP 200
5	<code>POST /api/auth/login</code> com credencial válida	token JWT no corpo
6	<code>POST /api/auth/login</code> com senha errada	HTTP 401
7	<code>POST /api/normas</code> sem token	HTTP 401

3.5 Workflow auxiliar — validação dos manifests Kubernetes

O arquivo `.github/workflows/k8s-validate.yml` roda a cada alteração em `k8s/`: renderiza os dois overlays com `kubectl kustomize` e valida os manifests gerados com `kubeconform -strict` contra os schemas oficiais do Kubernetes 1.29 — erros de manifesto aparecem no pull request, não no `apply`.

4. Docker: arquitetura, comandos e imagem

4.1 Arquitetura da imagem (multi-stage)



4.2 Estratégias adotadas

Estratégia	Motivação
Multi-stage build	O SDK não vai para a imagem final: apenas o runtime ASP.NET e os binários publicados, reduzindo drasticamente tamanho e superfície de ataque.
Camada de restore isolada	Os <code>.csproj</code> são copiados antes do código-fonte; enquanto as dependências não mudam, o <code>dotnet restore</code> vem do cache.
Stage <code>test</code> dedicado	<code>docker build --target test .</code> executa os testes dentro do container, com o mesmo resultado em qualquer máquina.
Usuário não-root	<code>appuser</code> (uid 1001) e <code>allowPrivilegeEscalation: false</code> no Kubernetes: o processo não roda como root.
<code>HEALTHCHECK</code> nativo	O Docker sabe se a API está saudável; o Compose usa isso em <code>depends_on: service_healthy</code> .
Configuração por ambiente	A mesma imagem atende dev, staging e produção — só mudam as variáveis injetadas.
<code>.dockerignore</code> enxuto	<code>bin/</code> , <code>obj/</code> , <code>.git/</code> , <code>docs/</code> e <code>entrega/</code> ficam fora do contexto de build.
Cache <code>type=gha</code>	As camadas são reaproveitadas entre execuções do pipeline.
<code>ARG APP_VERSION</code> + labels OCI	A versão implantada é visível no endpoint <code>/health</code> e nos metadados da imagem.

4.3 Comandos utilizados

```
# build da imagem, com a versão gravada nos metadados
docker build -t govambiental-api:local --build-arg APP_VERSION=1.0.0 .

# executar a bateria de testes dentro do container
docker build --target test .

# subir o ambiente completo (API + SQL Server)
docker compose up -d --build
docker compose ps
docker compose logs -f api

# staging e produção lado a lado, na mesma máquina
docker compose --env-file .env.staging -f docker-compose.staging.yml up -d
docker compose --env-file .env.prod -f docker-compose.prod.yml up -d

# inspecionar a saúde e a imagem
docker inspect --format '{{json .State.Health}}' govambiental-api-prod
docker images ghcr.io/<usuario>/govambiental-api

# publicar manualmente no registry (o pipeline faz isso automaticamente)
echo $GHCR_TOKEN | docker login ghcr.io -u <usuario> --password-stdin
docker tag govambiental-api:local ghcr.io/<usuario>/govambiental-api:v1.0.0
docker push ghcr.io/<usuario>/govambiental-api:v1.0.0

# encerrar
docker compose down          # preserva o volume do banco
docker compose down -v      # remove também os dados
```

4.4 Imagem criada

Item	Valor
Registry	ghcr.io (GitHub Container Registry)
Repositório da imagem	ghcr.io/gusttores/govambiental-api
Imagem base de build	mcr.microsoft.com/dotnet/sdk:8.0
Imagem base de runtime	mcr.microsoft.com/dotnet/aspnet:8.0
Tags geradas	sha-<commit> · main / develop · staging · latest · vX.Y.Z
Porta exposta	8080 (interna ao container, mapeada para 8080/8081 no host)
Usuário de execução	appuser (uid 1001, não-root)
Health check	curl -f http://localhost:8080/health a cada 15 s

4.5 Orquestração — Compose e Kubernetes

Arquivo	Ambiente	Porta	Destaques
docker-compose.yml	dev	8080	build local, banco exposto em 1433 para inspeção
docker-compose.staging.yml	staging	8081	imagem do GHCR, limites de recurso, rotação de logs, banco sem porta no host
docker-compose.prod.yml	produção	8080	restart: always, no-new-privileges, reservas de CPU/memória, senhas obrigatórias

Os três requisitos de composição do ambiente estão cobertos: **volumes** (volume nomeado por ambiente montado em `/var/opt/mssql`, preservando os dados entre `down` e `up`), **variáveis de ambiente** (tudo que muda entre ambientes vem de arquivos `.env`, com `${VAR:?mensagem}` falhando cedo se faltar uma senha) e **redes** (uma bridge dedicada por ambiente; a API acessa o banco pelo nome do serviço e, em staging e produção, o banco não publica porta no host).

Como alternativa completa, a pasta `k8s/` traz os manifests em Kustomize: `Deployment` com *rolling update* e três *probes*, `StatefulSet` do SQL Server com `PersistentVolumeClaim` de 5 Gi, `ConfigMap` e `Secret`, `Service`, `Ingress` e, em produção, `HorizontalPodAutoscaler` (2 a 5 réplicas por CPU) e `PodDisruptionBudget`, em namespaces separados por ambiente.

5. Evidências — pipeline em execução

Os prints abaixo comprovam as etapas de build, testes e deploy. Os arquivos de imagem ficam em docs/evidencias/ com os nomes indicados.

Figura 1 — Pipeline completo com todos os jobs concluídos

Aba **Actions** → execução do workflow “CI/CD - GovAmbiental API”, mostrando os 4 jobs encadeados em verde.

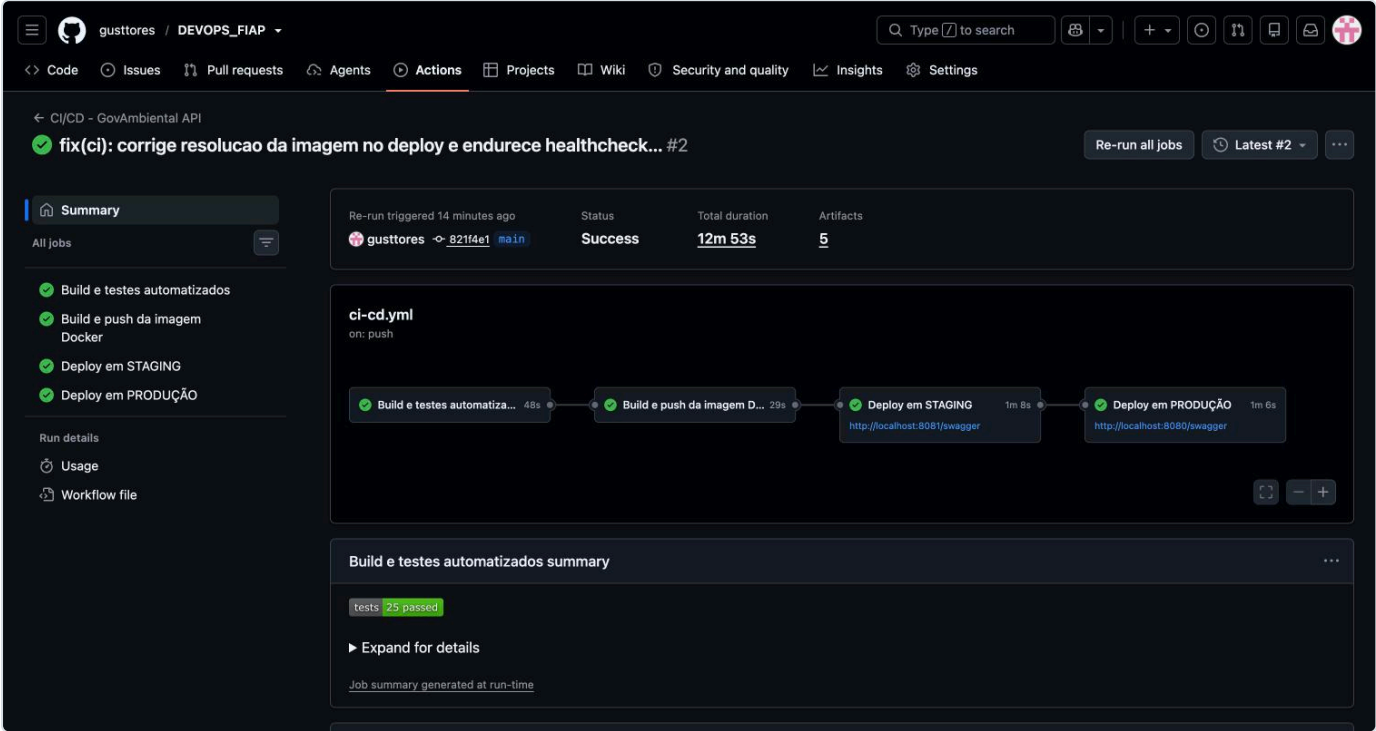


Figura 2 — Etapa de build e testes automatizados

Job Build e testes automatizados, com o passo `dotnet test` expandido mostrando os 25 testes aprovados.

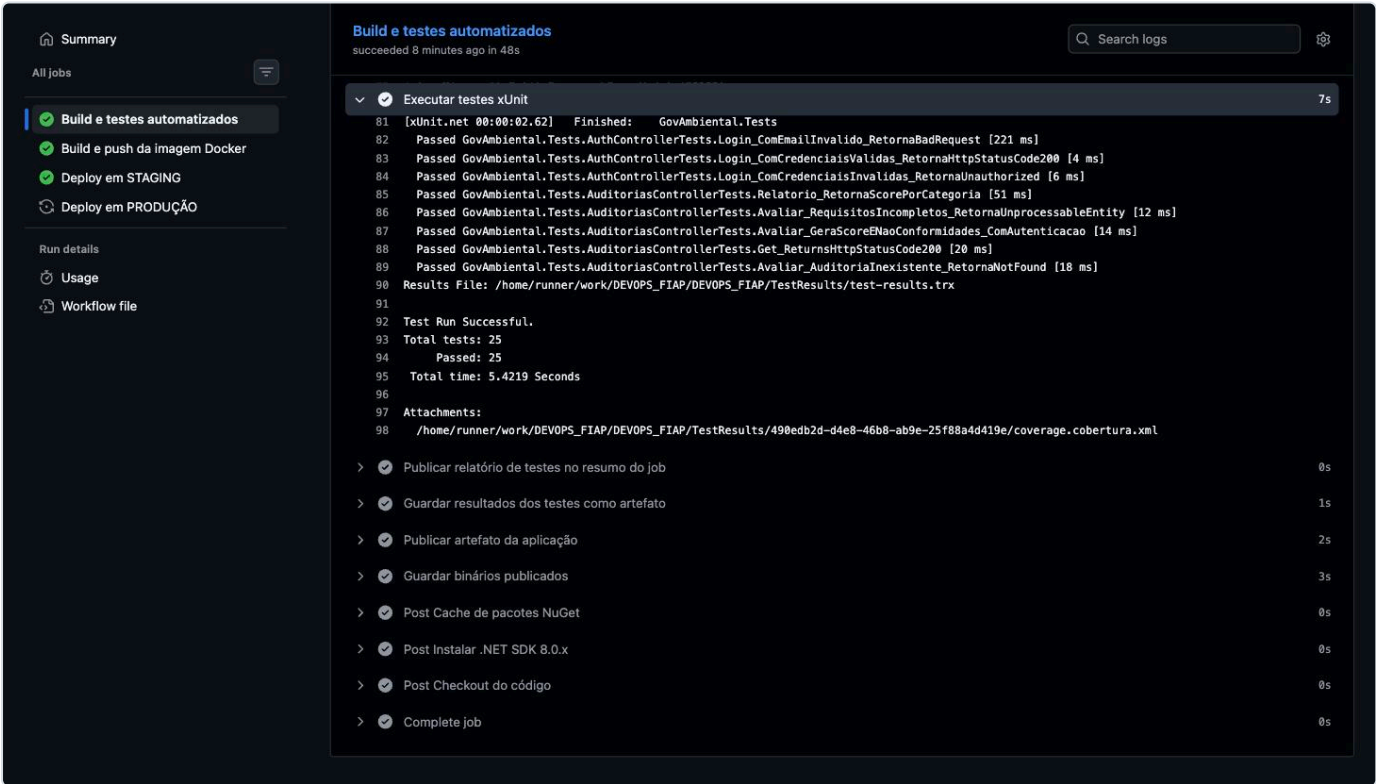


Figura 3 — Imagem Docker publicada no GHCR

Aba **Packages** do repositório (ou o resumo do job *Build e push da imagem Docker*) com as tags geradas.

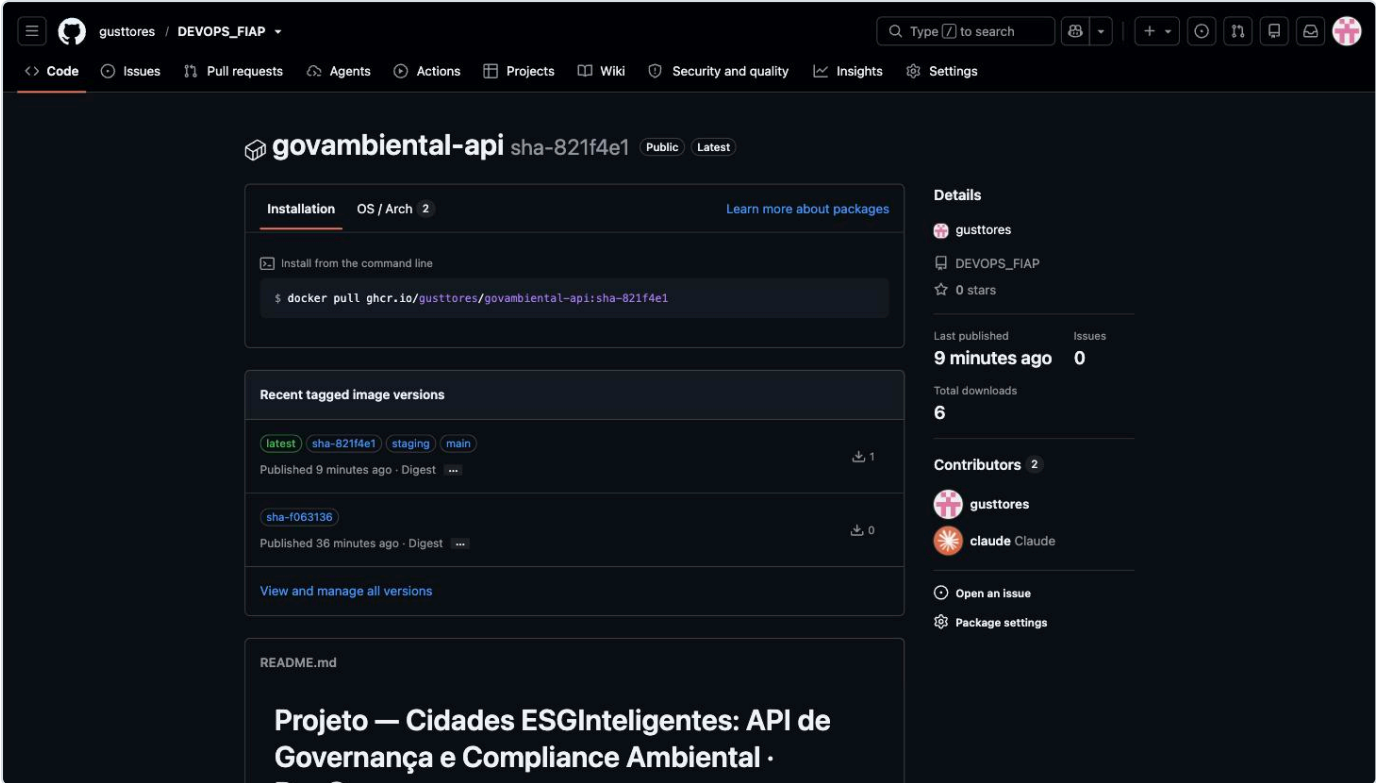


Figura 4 — Deploy em staging com smoke tests aprovados

Job **Deploy em STAGING**, passo *Smoke tests* expandido: “7/7 verificações passaram”.

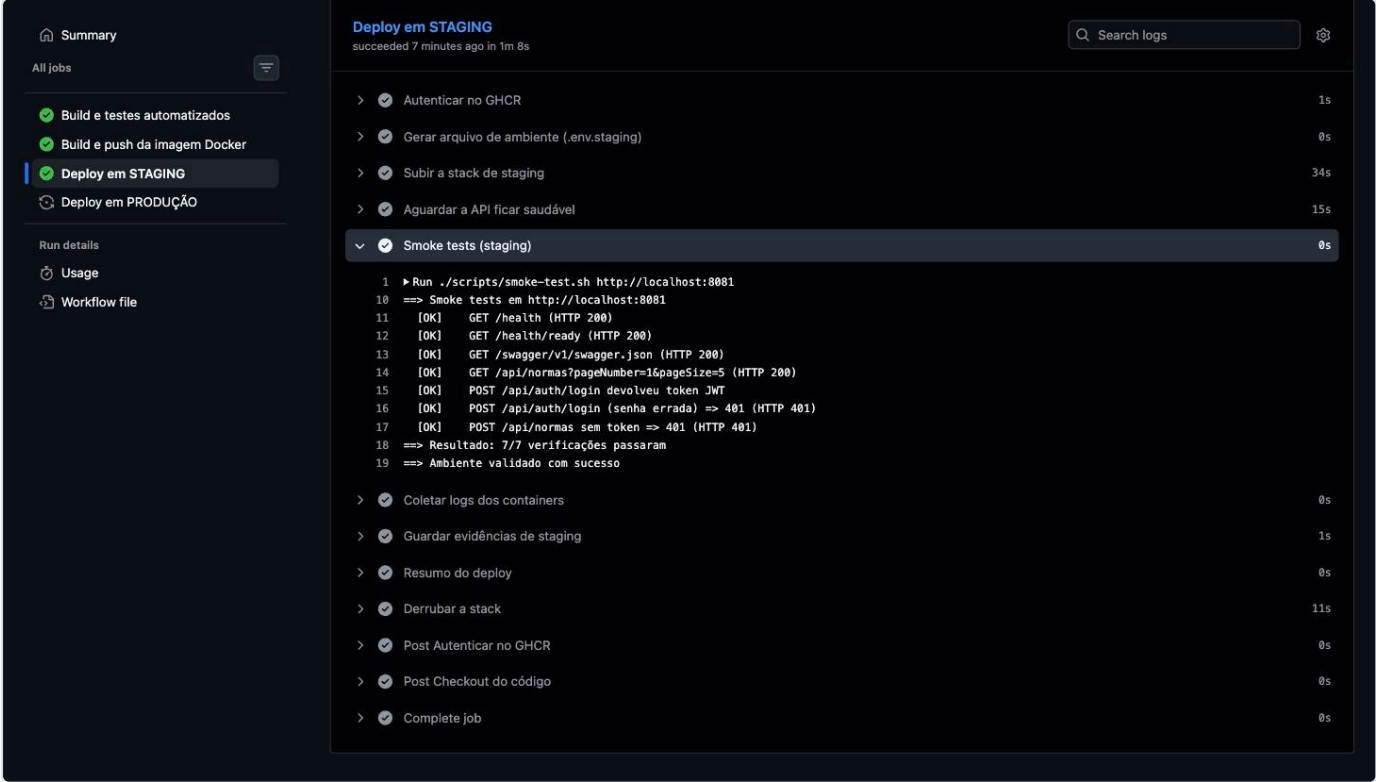


Figura 5 — Aprovação manual do deploy de produção

Tela *Review deployments* do Environment `production`, com o job aguardando aprovação.

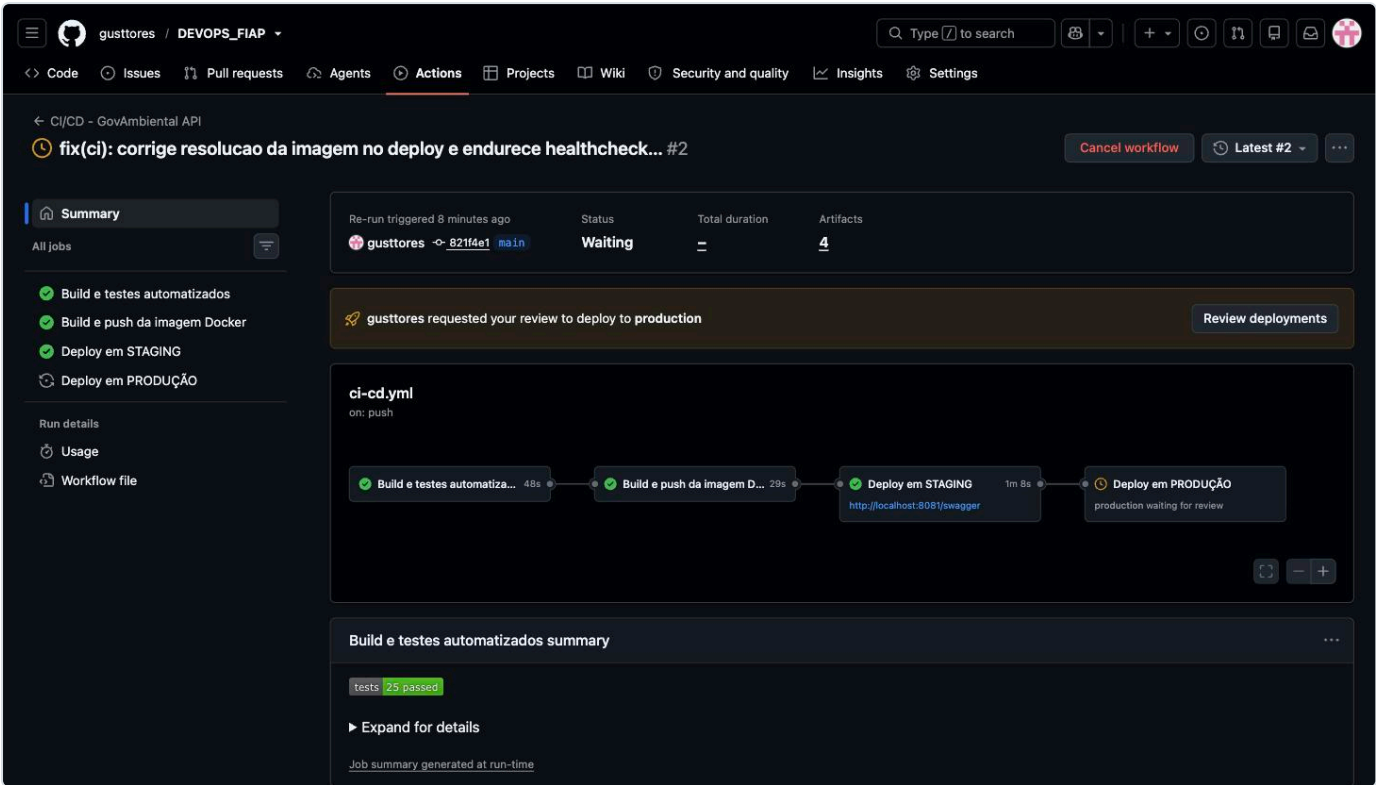
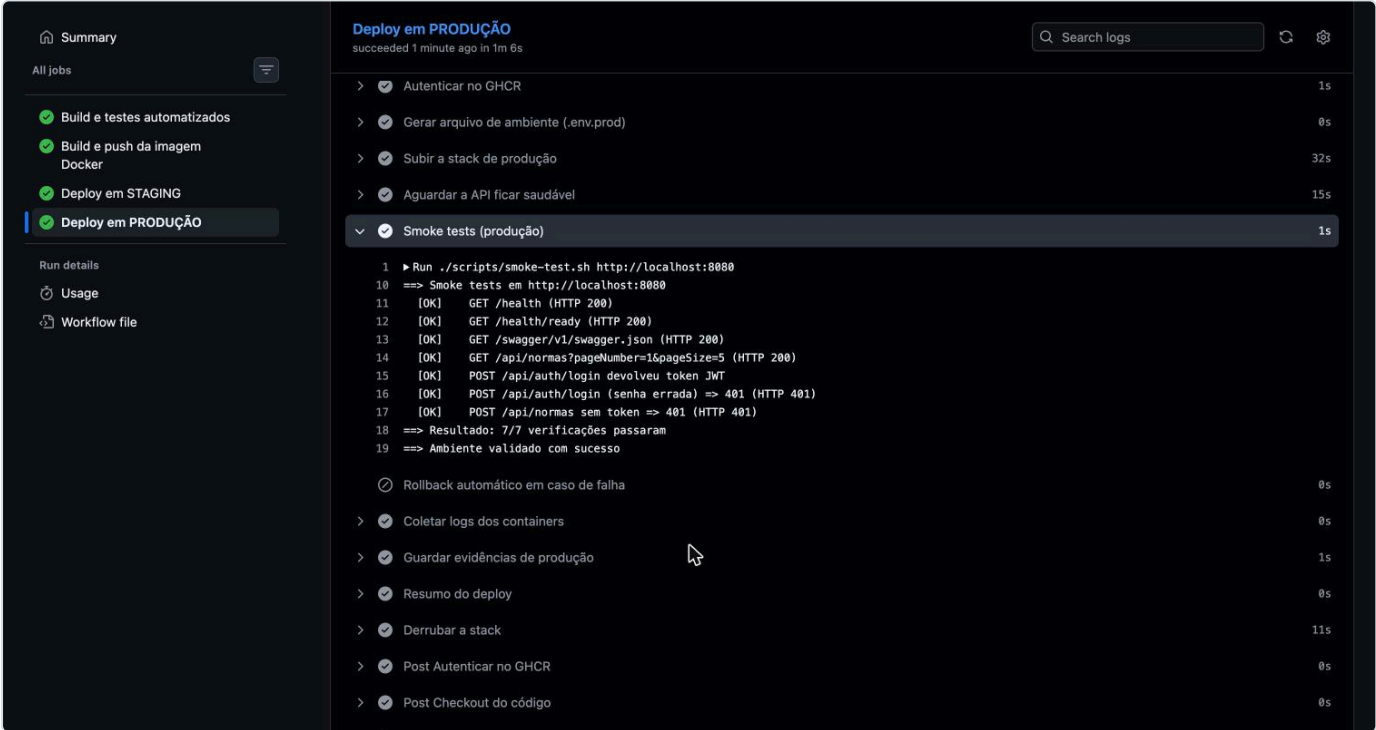


Figura 6 — Deploy em produção concluído

Job **Deploy em PRODUÇÃO** finalizado, com os smoke tests aprovados na porta 8080.



6. Evidências — ambientes em funcionamento

Figura 7 — Containers em execução e saudáveis

Terminal com `docker compose ps`: os serviços `api` e `db` com status `healthy`.

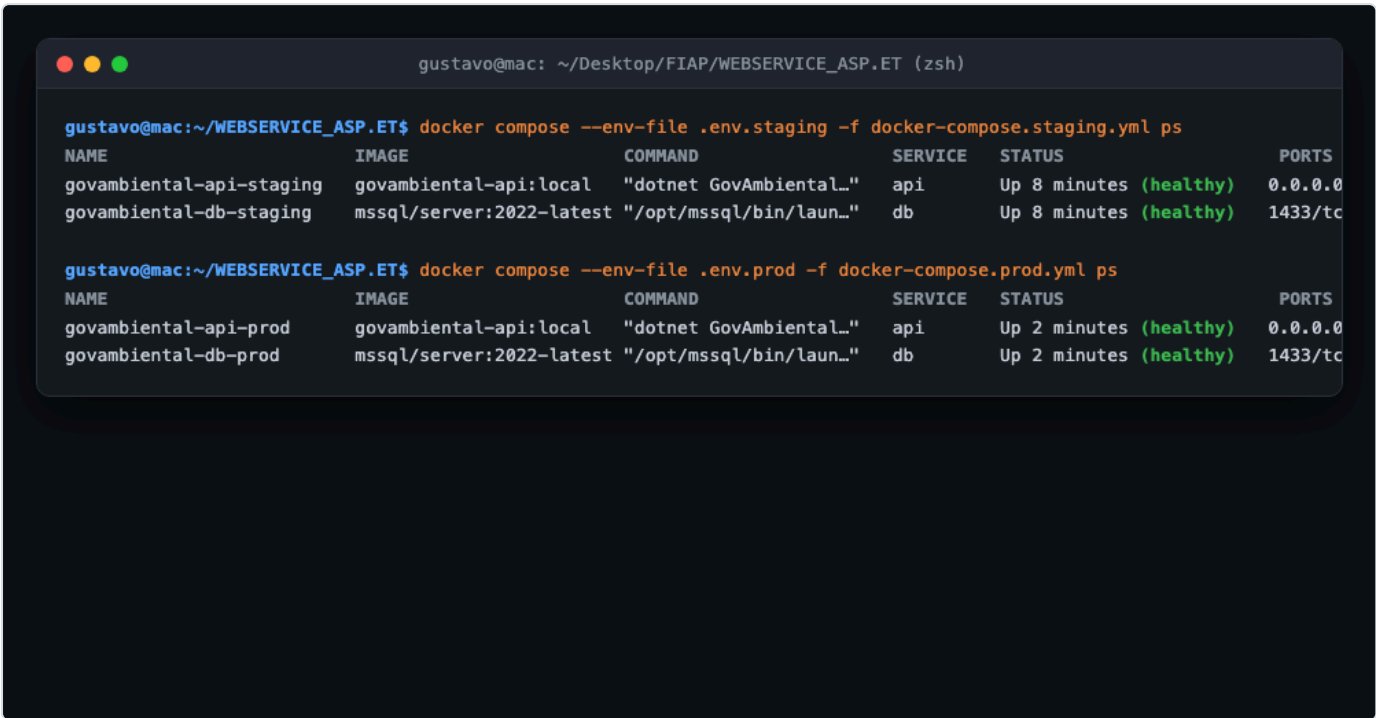


Figura 8 — Staging no ar (porta 8081)

Navegador em `http://localhost:8081/swagger` — Swagger da API rodando em staging.

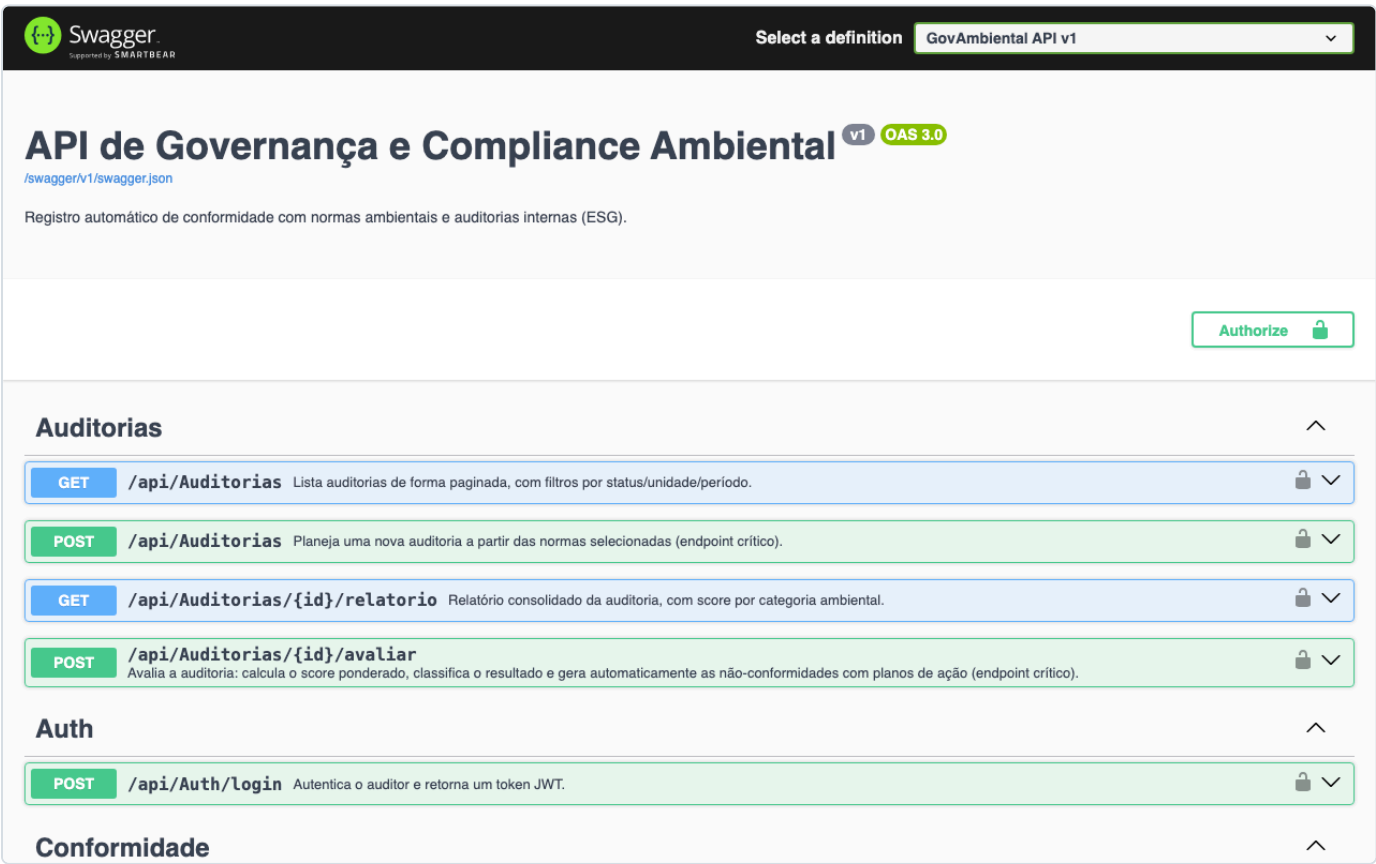


Figura 9 — Produção no ar (porta 8080)

Navegador em `http://localhost:8080/swagger` — mesma imagem, ambiente de produção.

 Swagger
Supported by SMARTBEAR

Select a definitionGovAmbiental API v1

API de Governança e Compliance Ambiental

v1OAS 3.0

/swagger/v1/swagger.json

Registro automático de conformidade com normas ambientais e auditorias internas (ESG).

Authorize

Auditorias

GET/api/AuditoriasLista auditorias de forma paginada, com filtros por status/unidade/período.

POST/api/AuditoriasPlaneja uma nova auditoria a partir das normas selecionadas (endpoint crítico).

GET/api/Auditorias/{id}/relatorioRelatório consolidado da auditoria, com score por categoria ambiental.

POST/api/Auditorias/{id}/avaliarAvalia a auditoria: calcula o score ponderado, classifica o resultado e gera automaticamente as não-conformidades com planos de ação (endpoint crítico).

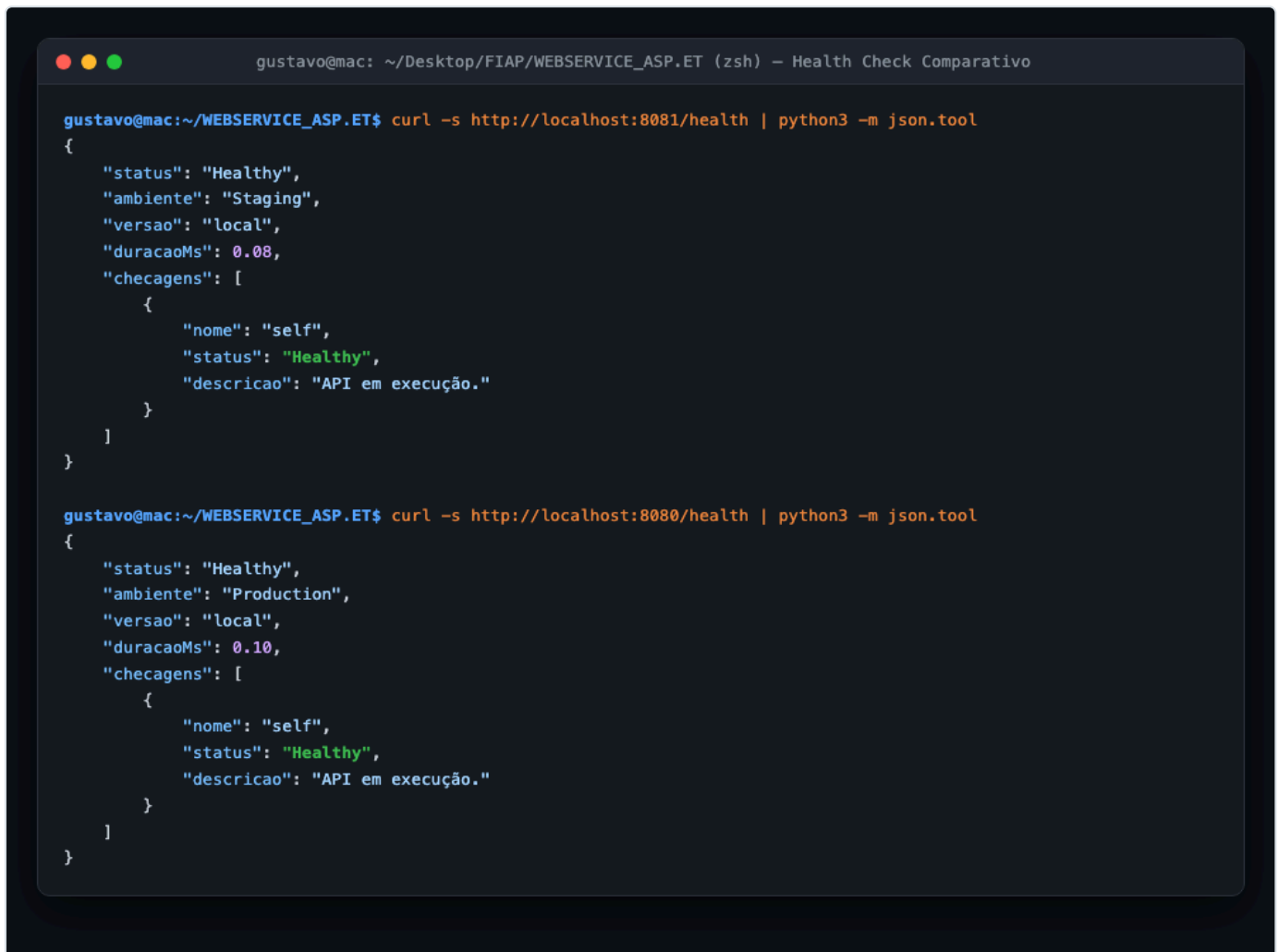
Auth

POST/api/Auth/loginAutentica o auditor e retorna um token JWT.

Conformidade

Figura 10 — Health check respondendo nos dois ambientes

`curl http://localhost:8081/health` e `curl http://localhost:8080/health` lado a lado, mostrando o campo `ambiente` diferente em cada um.

A terminal window titled "gustavo@mac: ~/Desktop/FIAP/WEBSERVICE_ASP.ET (zsh) - Health Check Comparativo" displays two JSON responses from health check endpoints. The first response, from localhost:8081, is for the "Staging" environment. The second response, from localhost:8080, is for the "Production" environment. Both responses show a "Healthy" status and a "self" component description.

```
gustavo@mac:~/WEBSERVICE_ASP.ET$ curl -s http://localhost:8081/health | python3 -m json.tool
{
  "status": "Healthy",
  "ambiente": "Staging",
  "versao": "local",
  "duracaoMs": 0.08,
  "checagens": [
    {
      "nome": "self",
      "status": "Healthy",
      "descricao": "API em execução."
    }
  ]
}

gustavo@mac:~/WEBSERVICE_ASP.ET$ curl -s http://localhost:8080/health | python3 -m json.tool
{
  "status": "Healthy",
  "ambiente": "Production",
  "versao": "local",
  "duracaoMs": 0.10,
  "checagens": [
    {
      "nome": "self",
      "status": "Healthy",
      "descricao": "API em execução."
    }
  ]
}
```

Evidências automáticas. Além dos prints, cada execução do pipeline anexa artefatos de comprovação: resultados-testes (relatório .trx e cobertura), evidencias-staging e evidencias-producao (saída de docker compose ps , logs completos dos containers e o JSON do endpoint /health de cada ambiente).

7. Desafios encontrados e como foram resolvidos

Desafio	Solução aplicada
A API subia antes do SQL Server e quebrava ao aplicar as migrations, derrubando o container logo no start.	<code>healthcheck</code> no serviço <code>db</code> com <code>sqlcmd + depends_on: condition: service_healthy ; EnableRetryOnFailure</code> no EF Core; e um laço de retry com <code>backoff</code> de 5 s (10 tentativas) em volta do <code>MigrateAsync()</code> , com log de cada tentativa.
O Docker não sabia se a aplicação estava realmente pronta — o container aparecia “running” antes de a API aceitar requisições.	Criação dos endpoints <code>/health</code> (liveness, sem tocar no banco) e <code>/health/ready</code> (readiness, com <code>CanConnectAsync</code>), usando apenas o pacote de health checks já embutido no ASP.NET Core. Eles alimentam o <code>HEALTHCHECK</code> do Docker, o <code>service_healthy</code> do Compose e as três <i>probes</i> do Kubernetes.
“Passou no build” não significa “está no ar”: o pipeline poderia ficar verde com um ambiente quebrado.	Smoke tests pós-deploy em Bash (<code>scripts/smoke-test.sh</code>) cobrindo 7 cenários reais — incluindo autenticação JWT e rejeição de acesso sem token. O job falha se qualquer verificação falhar.
Segredos não podem ir para a imagem nem para o Git, mas a aplicação precisa deles em tempo de execução.	Configuração inteiramente por variável de ambiente (<code>Jwt__Key</code> , <code>ConnectionStrings__DefaultConnection</code>), arquivos <code>.env</code> no <code>.gitignore</code> , <code>.env.example</code> versionado como modelo, GitHub Secrets no pipeline e <code>Secret</code> no Kubernetes.
Staging e produção colidiam ao rodar na mesma máquina: mesma porta, mesmo volume, mesmo nome de container.	Isolamento completo por ambiente — <code>name:</code> de projeto, rede bridge, volume nomeado, banco de dados, nome de container e porta próprios (8081 × 8080). Os dois ambientes rodam simultaneamente sem interferência.
Deploy automático em produção é arriscado demais para uma aplicação de compliance.	GitHub Environment <code>production</code> com <i>required reviewers</i> : o pipeline pausa e aguarda aprovação humana. Em caso de falha nos smoke tests, o passo <code>if: failure</code> faz rollback automático para a tag <code>latest</code> .
Build lento da imagem a cada commit, repetindo restore e download de pacotes.	Multi-stage com camada de restore separada, cache de camadas do Buildx (<code>type=gha</code>) e cache de pacotes NuGet no runner com chave pelo hash dos <code>.csproj</code> .
A imagem do SQL Server não roda nativamente em Apple Silicon, quebrando a execução local no Mac.	<code>platform: \${DB_PLATFORM:-linux/amd64}</code> nos três arquivos Compose: emulação no Mac, execução nativa no runner Linux do GitHub — o mesmo arquivo serve aos dois cenários.
Erros nos manifests do Kubernetes só apareciam na hora do <code>apply</code> , já no cluster.	Workflow <code>k8s-validate.yml</code> : <code>kubectl kustomize</code> renderiza os dois overlays e <code>kubeconform -strict</code> valida contra os schemas oficiais do Kubernetes 1.29, a cada push em <code>k8s/</code> .

8. Checklist de entrega

#	Item obrigatório	Status	Onde está
1	Projeto compactado em <code>.ZIP</code> com estrutura organizada	✓	<code>entrega/govambiental-devops.zip</code> — inclui <code>src/</code> , <code>tests/</code> , <code>Dockerfile</code> , arquivos <code>Compose</code> , <code>k8s/</code> , <code>.github/workflows/</code> , <code>scripts/</code> , <code>.env.example</code> , <code>README.md</code> e <code>docs/</code>
2	<code>Dockerfile</code> funcional	✓	<code>Dockerfile</code> — multi-stage (<code>restore</code> · <code>build</code> · <code>test</code> · <code>publish</code> · <code>final</code>), usuário não-root, <code>HEALTHCHECK</code> e labels OCI
3	<code>docker-compose.yml</code> ou arquivos Kubernetes	✓	Ambos: <code>docker-compose.yml</code> , <code>docker-compose.staging.yml</code> , <code>docker-compose.prod.yml</code> e <code>k8s/base/</code> + <code>k8s/overlays/{staging,production}/</code>
4	Pipeline com etapas de build, teste e deploy	✓	<code>.github/workflows/ci-cd.yml</code> — 4 jobs: <i>build-and-test</i> → <i>docker-image</i> → <i>deploy-staging</i> → <i>deploy-production</i>
5	<code>README.md</code> com instruções e prints	✓	<code>README.md</code> — 13 seções, incluindo execução local com Docker, pipeline, containerização, evidências e tecnologias
6	Documentação técnica com evidências (PDF ou PPT)	✓	Este documento — <code>docs/documentacao-tecnica.pdf</code>
7	Deploy realizado nos ambientes staging e produção	✓	Jobs <i>Deploy em STAGING</i> (porta 8081) e <i>Deploy em PRODUÇÃO</i> (porta 8080), ambos com smoke tests — evidências nas figuras 4 a 10

8.1 Tecnologias utilizadas

Camada	Stack
Aplicação	C# 12 · ASP.NET Core 8 · EF Core 8 · FluentValidation 11 · JWT Bearer · Swagger/OpenAPI
Banco de dados	Microsoft SQL Server 2022 (container) · migrations EF Core · EF Core InMemory nos testes
Testes	xUnit 2.9 · <code>Microsoft.AspNetCore.Mvc.Testing</code> · Coverlet
Containerização	Docker (multi-stage, não-root, HEALTHCHECK) · Docker Buildx · <code>.dockerignore</code>
Orquestração	Docker Compose v2 (3 ambientes) · Kubernetes + Kustomize (PVC, ConfigMap, Secret, Ingress, HPA, PDB)
CI/CD	GitHub Actions · GitHub Container Registry · Environments com aprovação · caches de NuGet e de camadas · artefatos de evidência
Qualidade e operação	Health checks ASP.NET Core · smoke tests em Bash · kubeconform · rollback automático